

Вариативная самостоятельная работа №2.2

АНАЛИТИЧЕСКАЯ ПОДБОРКА МАТЕРИАЛОВ ПО ЯЗЫКУ ПРОГРАММИРОВАНИЯ JULIA

Аннотация

Настоящим отчет представляет собой аналитическую подборку материалов по языку программирования Julia. Подборка включает аннотированные статьи и ресурсы, а также примеры решения задач с выдержками кода (бордами) и комментариями. Отчет предназначен для разработчиков, исследователей и студентов, интересующихся применением Julia в научных вычислениях, анализе данных и высокопроизводительных вычислениях.

ВВЕДЕНИЕ

Язык программирования Julia был создан в 2009 году и с тех пор приобрёл значительную популярность в научном сообществе благодаря сочетанию высокой производительности и удобства написания кода. По состоянию на февраль 2021 года Julia была загружена более 25 миллионов раз, причём за год прирост составил 87%.

Цель настоящего отчета — составить структурированную подборку материалов по языку Julia, включающую аннотированные статьи, официальные ресурсы и практические примеры решения задач. Основное внимание уделяется демонстрации ключевых возможностей языка через выдержки кода с комментариями и пояснениями.

АННОТИРОВАННЫЕ СТАТЬИ И РЕСУРСЫ ПО ЯЗЫКУ JULIA

Официальная документация Julia

Официальная документация языка Julia является основным ресурсом для изучения синтаксиса и стандартной библиотеки. Документация охватывает все аспекты языка: от базового синтаксиса до метапрограммирования и разработки на уровне внутренностей компилятора. Структура документации включает разделы «Manual», «Standard Library» и «Developer Documentation».

Ключевые разделы: Getting Started, Variables, Functions, Control Flow, Types, Methods, Metaprogramming, Multi-dimensional Arrays, Parallel Computing, Performance Tips.

Практическая ценность: Документация доступна для всех версий языка начиная с 0.3, что позволяет разработчикам работать с актуальными и историческими версиями.

Введение в язык Julia: обзор возможностей

Статья представляет собой обзорное введение в Julia, ориентированное на разработчиков, знакомых с Python или R. Автор описывает ключевые преимущества языка: высокую производительность, сравнимую с C/C++, и удобство, аналогичное Python.

Основные положения:

- Julia использует JIT-компиляцию на базе LLVM, что обеспечивает производительность, близкую к компилируемым языкам;
- Язык изначально спроектирован для параллельных вычислений;

- Экосистема включает библиотеки для машинного обучения: Flux.jl, MLJ.jl, MXNet.jl, Knet.jl;
- Пакет DataFrames.jl является аналогом pandas для табличных данных.

Достоинство статьи: Практический обзор с примерами применения в машинном обучении. Недостаток: отсутствие глубокого анализа недостатков языка.

Julia для анализа физиологических сигналов

Мини-обзор, посвящённый применению Julia для воспроизводимого и эффективного анализа физиологических сигналов. Авторы аргументируют выбор Julia для научных исследований.

Ключевые тезисы:

- Julia обеспечивает высокую производительность при сохранении интерактивности, что ускоряет как исследования, так и разработку программного обеспечения;
- Подавляющее большинство Julia-пакетов имеют открытый исходный код и разрабатываются на публичных платформах, что способствует воспроизводимости;
- Менеджер пакетов Julia позволяет создавать самодостаточные воспроизводимые проекты, устанавливаемые независимо от архитектуры процессора и операционной системы.

Практическая ценность: Статья демонстрирует применение Julia в конкретной предметной области (физиология) и может служить шаблоном для аналогичных обзоров в других областях.

Julia в физике высоких энергий

Технический обзор, описывающий применение Julia в задачах физики высоких энергий. Автор подробно рассматривает особенности языка, релевантные для научных вычислений.

Особенности языка, описанные в статье:

- Множественная диспетчеризация (multiple dispatch): функции специализируются по типам всех аргументов, а не только первого, что позволяет эффективно обрабатывать различные комбинации типов;
- Система типов: иерархическая структура типов позволяет создавать обобщённые функции, работающие с абстрактными типами (например, `AbstractArray{T,2}` для двумерных массивов любого типа);

- Оптимизация циклов: в отличие от Python, явные циклы в Julia оптимизируются эффективно и не уступают по скорости генераторам списков;
- Инструменты профилирования: макросы `@code_lowered`, `@code_native`, `@benchmark` позволяют анализировать производительность на уровне байт-кода и машинного кода.

Практическая ценность: Статья содержит подробное описание внутренних механизмов языка, что полезно для разработчиков, стремящихся к максимальной производительности.

Ресурсы для высокопроизводительных вычислений

Курируемая подборка материалов по Julia в контексте HPC (High-Performance Computing). Включает ссылки на воркшопы, tutorиалы и презентации.

Содержание подборки:

- Воркшопы по GPU-программированию на Julia (2021–2023);
- Tutorиалы по Julia для HPC на конференциях SC22, SC24;
- Материалы по Julia на архитектуре A64FX;
- Ежемесячные звонки сообщества JuliaHPC.

Практическая ценность: Актуальная подборка для разработчиков, работающих с параллельными вычислениями и GPU.

2 ПРИМЕРЫ РЕШЕНИЯ ЗАДАЧ НА ЯЗЫКЕ JULIA

Задача о факториале

Постановка: написать функцию `my_factorial(n)`, вычисляющую факториал числа `n` с использованием цикла `for`.

Листинг кода представлен ниже:

```
function my_factorial(n)
    result = one(n) # Инициализация значением 1 того же типа, что и n
    for i in 2:n
        result *= i
    end
    return result
end
```

Тестирование

```
my_factorial(4)    # 24
```

```
my_factorial(15)   # 1307674368000
```

```
my_factorial(big(30)) # 2652528598121910586363084800000000
```

Комментарии к коду:

- `one(n)` создаёт значение 1 того же типа, что и `n`. Это критически важно для поддержки `BigInt`: без этого при переполнении `Int64` результат станет отрицательным;
- Julia автоматически выполняет JIT-компиляцию функции при первом вызове, что обеспечивает высокую производительность при последующих вызовах.

Задача о биномиальном распределении

Написать функцию `binomial_rv(n, p)`, моделирующую одно испытание биномиального распределения `Bin(n, p)` с использованием только `rand()`.

Листинг кода представлен ниже:

```
function binomial_rv(n, p)
```

```
    successes = 0
```

```
    for i in 1:n
```

```
        if rand() < p
```

```
            successes += 1
```

```
        end
```

```
    end
```

```
    return successes
```

```
end
```

Пример: 10 бросков монеты с вероятностью успеха 0.5

```
bs = [binomial_rv(10, 0.5) for j in 1:10]
```

Комментарии к коду:

- `rand()` генерирует равномерно распределённое случайное число в интервале $[0, 1)$;
- Условие `rand() < p` моделирует успех с вероятностью `p`;

- Конструкция `[binomial_rv(10, 0.5) for j in 1:10]` демонстрирует генератор списков (comprehension), характерный для Julia.

Задача о матрице Странга

создать матрицу Странга $N \times N$ — трёхдиагональную матрицу с -2 на главной диагонали, 1 на смежных диагоналях и 0 в остальных позициях.

Листинг кода представлен ниже:

```
N = 10
```

```
A = zeros(N, N)
```

```
for i in 1:N
```

```
    A[i, i] = -2
```

```
    if i > 1
```

```
        A[i, i-1] = 1
```

```
        A[i-1, i] = 1
```

```
    end
```

```
end
```

Комментарии к коду:

- `zeros(N, N)` создаёт матрицу $N \times N$, заполненную нулями;
- Julia использует индексацию с единицы (1-based indexing), в отличие от C/C++ или Python;
- Матрица Странга возникает при дискретизации одномерного оператора Лапласа и часто используется в численных методах.

Задача о рюкзаке (целочисленная оптимизация)

Решить задачу о бинарном рюкзаке с использованием пакета `MathOptInterface.jl`:

$$\max c^T x \quad \text{при} \quad \omega^T x \leq C, \quad x_i \in \{0, 1\}$$

Листинг кода представлен ниже:

```
import MathOptInterface as MOI
```

```
using GLPK
```

```
optimizer = GLPK.Optimizer()
```

```
# Данные задачи
```

```
c = [1.0, 2.0, 3.0] # Ценности предметов
w = [0.3, 0.5, 1.0] # Веса предметов
C = 3.2             # Вместимость рюкзака
# Добавление переменных
x = MOI.add_variables(optimizer, length(c))
# Целевая функция (максимизация)
MOI.set(optimizer,
MOI.ObjectiveFunction{MOI.ScalarAffineFunction{Float64}}(),
    MOI.ScalarAffineFunction(MOI.ScalarAffineTerm.(c, x), 0.0))
MOI.set(optimizer, MOI.ObjectiveSense(), MOI.MAX_SENSE)
# Ограничение по вместимости
MOI.add_constraint(optimizer,
    MOI.ScalarAffineFunction(MOI.ScalarAffineTerm.(w, x), 0.0),
    MOI.LessThan(C))
# Ограничения целочисленности (0 или 1)
for x_i in x
    MOI.add_constraint(optimizer, x_i, MOI.ZeroOne())
end
# Решение
MOI.optimize!(optimizer)
# Получение результатов
status = MOI.get(optimizer, MOI.TerminationStatus()) # OPTIMAL
obj_value = MOI.get(optimizer, MOI.ObjectiveValue()) # 6.0
solution = MOI.get(optimizer, MOI.VariablePrimal(), x) # [1.0, 1.0, 1.0]
```

Комментарии к коду:

- `MOI.ScalarAffineTerm.(c, x)` использует broadcast-синтаксис Julia (точка после имени функции) для поэлементного применения операции;
- Задача решается с помощью GLPK — открытого решателя для задач линейного и целочисленного программирования;
- Оптимальное решение: все три предмета выбраны, суммарная ценность 6.0, суммарный вес 1.8 (не превышает 3.2).

ЗАКЛЮЧЕНИЕ

В ходе подготовки аналитической подборки были рассмотрены ключевые ресурсы по языку программирования Julia, включая официальную документацию, научные статьи и практические руководства. Проведённый анализ позволяет сделать следующие выводы.

Julia представляет собой язык, сочетающий высокую производительность (сопоставимую с C/C++) и удобство написания кода (близкое к Python). Ключевыми особенностями языка являются множественная диспетчеризация, развитая система типов и эффективная JIT-компиляция на базе LLVM.

Экосистема Julia включает библиотеки для машинного обучения (Flux.jl, MLJ.jl), анализа данных (DataFrames.jl) и оптимизации (MathOptInterface.jl). Менеджер пакетов обеспечивает воспроизводимость проектов, что особенно ценно для научных исследований.

Приведённые примеры решения задач — от базовых (факториал) до прикладных (целочисленная оптимизация) — демонстрируют гибкость языка и его применимость в различных предметных областях.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Datseris G., Zelko J. S. Physiological signal analysis and open science using the Julia language and associated software // Frontiers in Network Physiology. — 2024. — Vol. 4. — DOI: 10.3389/fnetp.2024.1478280.
2. Getting started with the Julia language // Grape Up. — 2026.
3. Julia Documentation. — URL: <https://docs.julialang.org>.
4. Julia in HEP // CERN Document Server. — 2025.
5. Julia Modeling Workshop // GitHub: SciML. — 2025.
6. MathOptInterface Documentation. — URL: <https://jump.dev/MathOptInterface.jl>.
7. Resources // JuliaParallel. — URL: <https://juliaparallel.org/resources>.